서울시립대학교 · 전자전기컴퓨터설계실험II

LAB6 실험 후 보고서

FPGA6 / MCU2 (STM32_2) — 서보모터 · SRAM 읽기·쓰기 · DAC · 컴퓨터와 FPGA 통신 (제공 예제 2개 · 과제5 · 과제6 · 과제7)

이름
형종희

학번
2023440145

조
C조

실험일
2026-10-26

작성일
2026-10-26

FPGA 보드
Combo II-DLD S75 · xc7s75fgga484-1

MCU 보드
NUCLEO-G474RE

도구
Vivado 2026.1 · Icarus Verilog 12.0 · STM32Cube (GCC 14.3.1) · VS Code Serial Monitor · Tektronix TBS2000B

📦 GitHub 제출 정보

항목	값
저장소	https://github.com/hyeongjonghee/electronics2-LABs (Public)
제출 태그	`lab6-submit` — 이 보고서(HTML · PDF)와 시연 영상 9개가 들어간 최종 커밋을 가리킴
이전 커밋	실험 전 보고서 태그 `lab6-prelab` (`366f5dd`) · 설계 소스와 시뮬레이션 `618512b` · 실험 당일 수정(서보 핀 · SRAM 핀/최상위 · 과제6·7 인터럽트판)과 영상은 `lab6-submit` 바로 앞 커밋
LAB6 경로	`LAB6/circuits/LAB6-01~04/` (src · sim · constraints · results), `LAB6/stm32/` (ex1 · ex2 · task5 · task6 · task7, 첫 판 `task6_v1_single` · `task7_v1_single` , PC 모의 실행 `host_mock`), `LAB6/evidence/videos/`
STM32 기반 프로젝트	`Glaysia/ece2_474` v1.1.0 = `dd94f01` . 학생이 바꾼 파일은 `Core/Src/ece2_474_routine.c` 하나

0. 결과 요약

10/26 실습에서 STM32_2의 제공 예제 2개 · 과제 3개와 FPGA 개별 실습 3개를 모두 보드에서 동작시키고, 실험마다 영상을 하나씩 남겼다(총 9개). 진행은 수업 매뉴얼 순서대로 했다. 먼저 컴퓨터 ↔ MCU 입출력(예제 ① · ② · 과제5)을 하고, MCU → FPGA 통신(과제6 · 7)으로 넘어간 다음, 마지막에 FPGA 단독 실습(서보 → SRAM → DAC)을 했다.

#	실험 (무엇을 하는가)	사전 검증	교재 핀 대조	보드 빌드 (LUT · FF · IOB · WNS)	보드 결과
1	**예제 ①** getchar 되돌림 — PC가 보낸 글자를 MCU가 받아 다시 출력	모의 실행	—	—	**H · i ✔**
2	**예제 ②** UART3 송신 — 받은 글자를 PC10으로 FPGA 쪽에 내보내고 결과 코드 출력	모의 실행	—	—	**결과 0 · 파형 3.2 V ✔**
3	**과제5** PC → MCU → PC — 받은 글자와 그 바이트 값(16진수)을 출력	모의 실행	—	text 27,740 B	**0x31 · 0x41 · 0x61 ✔**
4	**과제6** PC → MCU → FPGA → PC — MCU가 넘긴 글자를 FPGA가 자기 USB-UART로 PC에 되돌려 보냄	공동 시뮬레이션	p.73 · F8 일치	142 · 127 · 12 · +14.748 ns	**여러 글자 지원으로 수정 후 ✔**
5	**과제7** UART 명령으로 FPGA LED — 글자 0 · 1 · 2를 끄기 · 켜기 · 반전 명령으로 해석	공동 시뮬레이션	과제6과 같음	과제6과 같은 비트스트림	**켜짐 유지 · 반전 ✔**
6	**서보모터** — 버튼으로 20 ms 주기 펄스의 폭(1.0~2.0 ms)을 바꿔 각도 제어	Icarus · XSim	실습 25 · AA22로 변경	411 · 125 · 13 · +13.281 ns	**각도 단계 변화 ✔**
7	**SRAM** — DIP로 정한 주소에 4비트 값을 쓰고 다시 읽어 LED로 확인	Icarus · XSim	p.603 · 37핀 작성, UB/LB 없음	11 · 8 · 56 · +16.899 ns	**쓰기 · 읽기 ✔**
8	**DAC** — 0~255를 차례로 써서 아날로그 톱니파 출력	Icarus · XSim	p.625 · 클록만 B6	22 · 62 · 23 · +14.854 ns	**톱니 2.56 s ✔**

실험 전 보고서 대비 바뀐 점

- **과제6 · 7 MCU 코드**: 조교 확인 때 "한 글자씩이 아니라 여러 글자를 한 번에 보내도 전달되어야 한다"는 요구를 받았다. getchar로 한 글자씩 받는 첫 판은 `"1Aa"`를 한 번에 보내면 앞 글자만 전달됐다. UART2 수신 인터럽트 + 256바이트 링 버퍼로 다시 만들어 `"hello, world!"` 같은 문자열이 그대로 FPGA까지 갔다(4장 · 고찰 3).
- **MCU → FPGA 연결 조건**: 실험 전에는 확장 포트의 PORT A ENABLE 스위치를 ON으로 계획했으나, 실제로는 **OFF여야 연결**된다(버스 스위치의 활성 LOW 입력). 또 보드의 CLOCK SELECT 다이얼이 50 MHz가 아니면 UART 속도가 맞지 않는다. 두 가지를 바로잡고서야 FPGA가 글자를 받았다(4장 ⑥).
- **서보**: 출력 핀을 임시 E8에서 교재 실습 25 핀표의 **AA22**로 바꾸어 다시 빌드했다(WNS +13.281 ns).
- **SRAM**: 교재 p.603 핀 구성표로 주소 18 · 데이터 16 · CS/WE/OE를 XDC에 채웠다. 칩은 Samsung **K6R4016V1D**이고 UB/LB는 표에 없다(보드에서 GND에 고정). 그래서 보드용 최상위에서 UB/LB 포트 두 개만 지우고, 버튼 동기화와 데이터 2클록 유지는 그대로 두었다.
- **DAC**: 미리 만든 비트스트림 그대로 동작했다. 보드의 DAC 출력 단자는 **D/A OUT 2** 하나뿐이라 그곳에서 측정했다.

목차

1. 예제 ① · ②
2. 과제5
3. 과제6 (여러 글자)
4. 과제7
5. 서보모터
6. SRAM
7. DAC
8. 고찰
9. 참고문헌

공통 측정 환경

- **PC 연결**: MCU는 ST-LINK 가상 COM 포트(**COM5**), FPGA 보드의 USB-UART는 FTDI FT232R(**COM6**). 둘 다 9600 bps · 8N1, Serial Monitor 줄바꿈 = 없음(None).
- **배선**: MCU PC10(UART3 TX) → FPGA 확장 포트 PORT A의 I/O 00 = **BDIO_00 (F8)**, MCU GND ↔ FPGA GND.
- **Serial Monitor의 한글 깨짐**: MCU가 보내는 한글은 UTF-8(글자당 3바이트)인데, 9600 bps에서는 바이트가 천천히 들어와 Serial Monitor가 한 글자의 3바이트를 서로 다른 덩어리로 나누어 해석한다. 그래서 영상에서 `받은 문자:` 같은 한글 부분이 ◆ 기호로 보인다. 영문 · 숫자 · 16진수는 1바이트라 그대로 보이며, 판단에 필요한 값은 모두 이 부분에 있다. MCU가 실제로 보낸 바이트는 정상이다(같은 코드의 PC 모의 실행 출력은 한글이 정상).

1. STM32_2 제공 예제 ① · ② — PC와 MCU의 문자 입출력

① 목적

예제 ①은 PC의 Serial Monitor에서 보낸 글자를 MCU가 `getchar()` 로 받아 그대로 다시 출력한다. 예제 ②는 받은 글자 한 바이트를 UART3(PC10)로 FPGA 쪽에 보내고, 송신 함수가 돌려준 결과 코드를 출력한다. 이후 과제 5 ~ 7은 모두 이 두 예제를 바탕으로 한다.

② 이론

`getchar()` 는 `HAL_UART_Receive(&huart2, …, HAL_MAX_DELAY)` 로 이어져, 글자가 올 때까지 그 자리에서 기다린다. 화면이 조용한 것은 멈춘 것이 아니라 기다리는 상태다. 예제 ②의 `HAL_UART_Transmit` 결과 `HAL_OK = 0` 은 "MCU가 마지막 비트까지 내보냈다"는 뜻일 뿐, 받는 쪽이 받았다는 확인이 아니다. UART에는 수신 확인 응답이 없기 때문이다.

$$t_{bit} = \frac{1}{9600} = 104.17\ \mu s, \qquad \text{한 글자} = 10\,t_{bit} = 1.042\ \text{ms}\ (\text{시작}\ 1 + \text{데이터}\ 8 + \text{정지}\ 1)$$

③ 코드 (polling_routine)

예제 ①

```
void polling_routine(void)
{
    int ch = getchar();
    if (ch != EOF)
    {
        printf("받은 문자: %c\r\n", ch);
    }
}
```

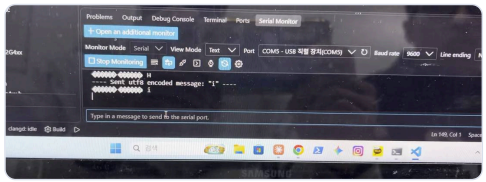
예제 ②

```
void polling_routine(void)
{
    int ch = getchar();
    if (ch != EOF)
    {
        uint8_t data = (uint8_t)ch;
        HAL_StatusTypeDef result =
            HAL_UART_Transmit(&huart3, &data, 1U,
100U);
        printf("전송 결과: %d\r\n", (int)result);
    }
}
```

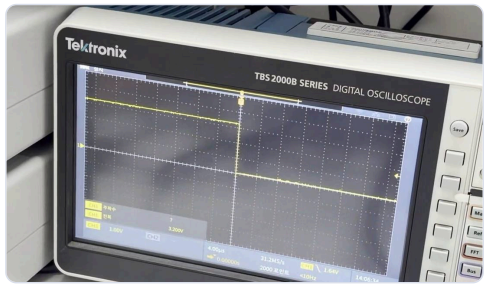
④ 보드 검증

확인 항목	기대	관찰	결과
예제 ① · H, i를 하나씩 전송	받은 문자: H / 받은 문자: i	Serial Monitor에 H, i가 차례로 출력(앞의 한글은 깨져 보임)	일치
예제 ② · Z 전송	전송 결과: 0	전송 결과 0 출력	일치
예제 ② · PC10 파형 (1 V/칸, 4 µs/칸, 하강 에지 트리거 1.64 V)	쉴 때 HIGH(3.3 V) → 시작 비트에서 LOW	HIGH에서 LOW로 떨어지는 시작 비트 에지, 진폭 측정값 3.20 V	일치

처음에는 진폭이 80 ~ 120 mV로 작게 보였다. 신호가 없는 동안의 잡음을 본 것으로, 하강 에지 트리거를 걸고 Single로 한 번만 잡자 3.2 V 크기의 시작 비트가 잡혔다. 비트 하나의 폭은 과제6에서 `'U'` (0x55, 0과 1이 번갈아 나오는 글자)로 따로 쟀다.



예제 ① · H와 i를 보내자 각각 되돌려 출력됨



예제 ② · PC10의 시작 비트 하강 에지 (진폭 3.200 V)

예제 ① 영상 (15.7 s)

영상: LAB6/evidence/videos/LAB6-01_ex1_getchar.mp4 (GitHub에서 열기 ↗)

예제 ② 영상 (8.3 s) · 노트북 화면의 전송 결과와 오실로스코프 파형

영상: LAB6/evidence/videos/LAB6-02_ex2_uart3.mp4 (GitHub에서 열기 ↗)

2. STM32_2 과제5 — PC → MCU → PC (글자와 바이트 값 출력)

① 목적

PC에서 보낸 글자 한 개를 MCU가 받아, 그 글자와 바이트 값(16진수)을 함께 출력한다. 키보드 글자가 실제로는 ASCII 숫자 한 바이트로 전송된다는 것을 확인한다.

② 이론

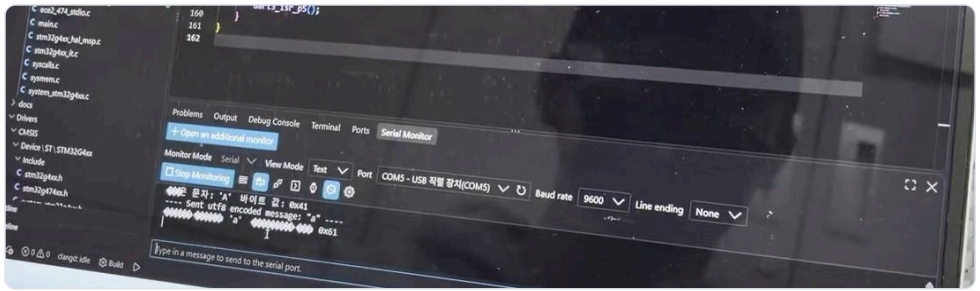
숫자 글자 `'0'~'9'` 는 0x30 ~ 0x39, 대문자는 0x41부터, 소문자는 대문자 + 0x20이다. 그래서 글자 `'1'` (0x31)과 숫자 1(0x01)은 서로 다른 값이다. `%02X` 는 16진수 대문자 두 자리로 찍는 형식이다. 화면에 찍히지 않는 제어 문자(0x20 미만 · 0x7F 이상)는 글자 대신 "(제어 문자)"로 표시한다.

③ 코드

```
void polling_routine(void)
{
    int ch = getchar();                 /* 입력이 올 때까지 여기서 기다림(멈춘 것이 아님) */
    if (ch != EOF)                      /* EOF(-1) = 수신 오류. 정상 문자만 처리 */
    {
        uint8_t data = (uint8_t)ch;     /* 0~255 한 바이트로 변환 */
        if (data >= 0x20U && data <= 0x7EU)
        {
            /* 화면에 찍히는 문자(공백~'~')는 문자와 값을 함께 출력 */
            printf("받은 문자: '%c'  바이트 값: 0x%02X\r\n", (char)data, (unsigned int)data);
        }
        else
        {
            /* 줄바꿈(0x0D, 0x0A) 같은 제어 문자는 값만 출력 */
            printf("받은 문자: (제어 문자)  바이트 값: 0x%02X\r\n", (unsigned int)data);
        }
    }
}
```

④ 보드 검증

입력	기대	관찰	결과
1	'1' · 0x31	'1' 0x31	일치
A	'A' · 0x41	'A' 0x41	일치
a	'a' · 0x61	'a' 0x61	일치



과제5 · 'A' → 바이트 값 0x41, 'a' → 0x61 (대문자와 소문자의 차이 0x20)

과제5 영상 (10.3 s)

영상: LAB6/evidence/videos/LAB6-03_task5_hex.mp4 (GitHub에서 열기 ↗)

3. STM32_2 과제6 — PC → MCU → FPGA → PC (여러 글자)

① 목적

PC가 MCU에 보낸 글자를 MCU가 UART3(PC10)로 FPGA에 넘기고, FPGA(`uart_cmd_top`)는 받은 글자를 자기 USB-UART(F6)로 PC에 다시 보낸다. PC에서 COM5(MCU)와 COM6(FPGA)를 함께 열어, 보낸 문자열과 FPGA가 받은 문자열이 같은지 비교한다. 조교 요구에 따라 **여러 글자를 한 번에 보내도** 빠짐없이 전달되어야 한다.

② 이론 — 왜 첫 판은 여러 글자를 놓쳤나

첫 판(아래 왼쪽)은 `getchar()`로 한 글자를 받은 뒤, UART3 전송(1 ms)과 결과 `printf`(약 40바이트 ≈ 42 ms)를 마치고 나서야 다음 글자를 받으러 간다. 그런데 Serial Monitor는 문자열을 쉬지 않고 보내므로 다음 글자가 1.04 ms 뒤에 도착한다. UART2 수신 레지스터는 한 바이트만 담으므로, 기다려 주는 쪽이 없으면 뒤 글자가 앞 글자를 덮어쓰고(오버런) HAL은 오류 상태가 된다. 그래서 `"1Aa"`를 한 번에 보내면 앞 글자만 전달됐다.

$$\underbrace{1.04\ \text{ms}}_{\text{다음 글자 도착}} \ll \underbrace{1.04\ \text{ms} + \approx 42\ \text{ms}}_{\text{첫 판이 한 글자에 쓰는 시간}}$$

수정판(v2)은 받기와 처리를 나누었다. ① UART2 수신 인터럽트가 바이트가 올 때마다 바로 링 버퍼(256바이트)에 넣고 다음 수신을 다시 요청한다(수 µs). ② 메인 루프는 버퍼에서 하나씩 꺼내 UART3로 보낸다. 들어오는 속도와 나가는 속도가 모두 9600 bps라 버퍼가 쌓이지 않는다. ③ 결과 출력은 20 ms 동안 새 바이트가 없을 때(한 묶음이 끝났을 때) 한 줄로만 한다. 오류가 나면 HAL이 수신을 멈추므로 `HAL_UART_ErrorCallback`에서 수신을 다시 시작한다.

링 버퍼의 `head`는 인터럽트만, `tail`은 메인 루프만 바꾼다. 8비트 인덱스는 한 번에 읽고 쓰이므로 두 쪽이 서로의 값을 깨뜨리지 않고, 256에서 자연히 0으로 돌아간다(가득 참 = `head+1 == tail`, 최대 255바이트).

③ 코드 (첫 판 → 수정판)

첫 판 · task6_v1_single

```
void polling_routine(void)
{
    int ch = getchar();
    if (ch != EOF)
    {
        uint8_t data = (uint8_t)ch;
        /* 1바이트 송신, 최대 100 ms 대기. 결과 0(HAL_OK)은 "MCU가 다 보냈다"는 뜻일 뿐,
         * FPGA가 받았는지는 FPGA 쪽 COM 포트 출력으로 따로 확인합니다. */
        HAL_StatusTypeDef result = HAL_UART_Transmit(&huart3, &data, 1U, 100U);
        if (data >= 0x20U && data <= 0x7EU)
        {
            printf("FPGA로 보냄: '%c' (0x%02X)  전송 결과: %d\r\n",
                   (char)data, (unsigned int)data, (int)result);
        }
        else
        {
            printf("FPGA로 보냄: (제어 문자) (0x%02X)  전송 결과: %d\r\n",
                   (unsigned int)data, (int)result);
        }
    }
}
```

수정판 · 링 버퍼와 수신 인터럽트

```
/* 여러 글자를 한 번에 받아도 놓치지 않도록 PC 수신 바이트를 모아 두는 링 버퍼(256바이트).
 * 인터럽트(uart2_isr_p6)가 넣고, 메인 루프(polling_routine)가 꺼냅니다.
 * head는 ISR만, tail은 메인 루프만 바꾸므로 8비트 인덱스 하나씩이면 충돌하지 않습니다. */
#define RX_BUF_SIZE 256U
static volatile uint8_t rx_buf[RX_BUF_SIZE];
static volatile uint8_t rx_head = 0U;   /* 다음에 넣을 위치 (ISR) */
static volatile uint8_t rx_tail = 0U;   /* 다음에 꺼낼 위치 (메인 루프) */
static volatile uint32_t rx_overflow = 0U;
```

```
static void uart2_isr_p6(void)
{
    /* 받은 1바이트를 링 버퍼에 넣고 바로 다음 수신을 요청합니다(짧게 끝냄). */
    uint8_t next = (uint8_t)(rx_head + 1U);
    if (next != rx_tail)
    {
        rx_buf[rx_head] = uart2_rx_ece2;
        rx_head = next;
    }
    else
    {
        rx_overflow++;   /* 버퍼가 가득 참: 이 바이트는 버림 */
    }
    HAL_UART_Receive_IT(&huart2, &uart2_rx_ece2, 1U);
}
```

```
/* 수신 중 오버런·잡음 등 오류가 나면 HAL이 수신을 멈추므로, 여기서 다시 시작합니다. */
void HAL_UART_ErrorCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART2)
    {
        HAL_UART_Receive_IT(&huart2, &uart2_rx_ece2, 1U);
    }
}
```

수정판 · polling_routine

```
void polling_routine(void)
{
    static char line[RX_BUF_SIZE + 1U];
    static uint32_t count = 0U;
    static uint32_t fail = 0U;
    static uint32_t last_ms = 0U;     /* 마지막으로 바이트를 꺼낸 시각 */

    if (rx_tail != rx_head)
    {
        uint8_t data = rx_buf[rx_tail];
        rx_tail = (uint8_t)(rx_tail + 1U);
        last_ms = HAL_GetTick();
        if (HAL_UART_Transmit(&huart3, &data, 1U, 100U) != HAL_OK)
        {
            fail++;
        }
        if (count < RX_BUF_SIZE)
        {
            /* 화면에 찍히지 않는 제어 문자(CR·LF 등)는 '.'로 표시 */
            line[count] = (data >= 0x20U && data <= 0x7EU) ? (char)data : '.';
            count++;
        }
    }
    else if (count > 0U && (HAL_GetTick() - last_ms) >= 20U)   /* 20 ms 동안 새 바이트 없음 = 한 묶음 끝 */
    {
        line[count] = '\0';
        printf("FPGA로 보냄: \"%s\" (%lu바이트, 전송 실패 %lu, 버퍼 넘침 %lu)\r\n",
               line, (unsigned long)count, (unsigned long)fail, (unsigned long)rx_overflow);
        count = 0U;
        fail = 0U;
    }
}
```

FPGA 회로 `uart_cmd_top`(수신 → 송신 되돌림 + 명령 해석)은 실험 전 보고서 6장 ④와 같고 수정하지 않았다.

④ 사전 검증 (실험 후 추가)

PC 모의 실행(가짜 HAL로 같은 소스를 gcc 컴파일)에 `Hello World`와 `1Aa`를 묶음으로 넣었다. 가짜 인터럽트가 바이트를 링 버퍼에 넣고 메인 루프가 꺼내도록 해, 묶음마다 한 줄이 나오는지 보았다.

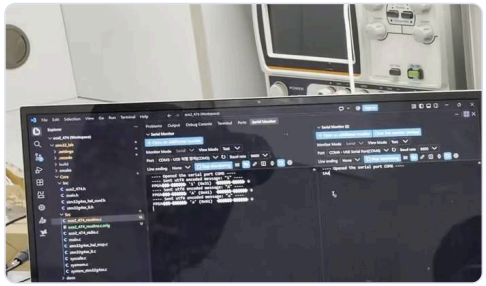
```
FPGA로 보냄: "Hello World" (11바이트, 전송 실패 0, 버퍼 넘침 0)
FPGA로 보냄: "1Aa" (3바이트, 전송 실패 0, 버퍼 넘침 0)
```

FPGA가 쉬는 시간 없이 이어 오는 바이트를 따라가는지도 확인했다. MCU가 UART3로 낸 14바이트를 정지 비트 직후 다음 시작 비트가 오도록 붙여서 보내는 TB(`sim/tb_b2b.sv`)를 만들었다. 비트 폭은 보드에서 잰 MCU 값 104.8 µs를 썼다(⑥). FPGA 수신기는 9600.6 bps라 MCU(약 9542 bps)보다 0.6 % 빠르지만, 매 글자의 시작 비트에서 다시 맞추므로 어긋남이 쌓이지 않는다. 14바이트 모두 순서대로 되돌아왔고 프레이밍 오류는 0이었다.

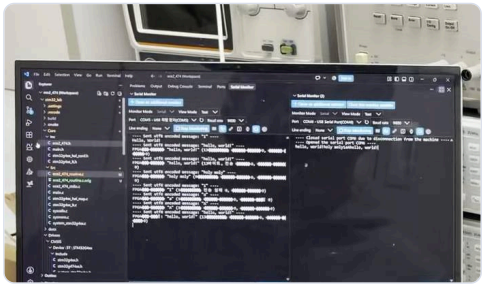
```
  2006.2 us  PC(FPGA COM) <- 0x48   LED1=0  LED2~8=1001000
  3054.2 us  PC(FPGA COM) <- 0x65   LED1=0  LED2~8=1100101
  4102.2 us  PC(FPGA COM) <- 0x6c   LED1=0  LED2~8=1101100
  5150.2 us  PC(FPGA COM) <- 0x6c   LED1=0  LED2~8=1101100
  6198.2 us  PC(FPGA COM) <- 0x6f   LED1=0  LED2~8=1101111
  7246.2 us  PC(FPGA COM) <- 0x20   LED1=0  LED2~8=0100000
  8294.2 us  PC(FPGA COM) <- 0x57   LED1=0  LED2~8=1010111
  9342.2 us  PC(FPGA COM) <- 0x6f   LED1=0  LED2~8=1101111
 10390.2 us  PC(FPGA COM) <- 0x72   LED1=0  LED2~8=1110010
 11438.2 us  PC(FPGA COM) <- 0x6c   LED1=0  LED2~8=1101100
 12486.2 us  PC(FPGA COM) <- 0x64   LED1=0  LED2~8=1100100
 13534.2 us  PC(FPGA COM) <- 0x31   LED1=1  LED2~8=0110001
 14582.2 us  PC(FPGA COM) <- 0x41   LED1=1  LED2~8=1000001
 15630.2 us  PC(FPGA COM) <- 0x61   LED1=1  LED2~8=1100001
PASS uart_cmd_top: 14 bytes echoed, LED1 follows 0/1/2 commands
sim/tb_b2b.sv:46: $finish called at 15950600000 (1ps)
```

⑤ 보드 검증

확인 항목	기대	관찰	결과
첫 판 · 1, A, a를 하나씩	COM6 = 0x31 0x41 0x61	FPGA 쪽 COM6에 1, A, a가 차례로 도착	일치
첫 판 · "1Aa"를 한 번에	세 글자 모두 도착	앞 글자만 도착 (조교 지적)	수정 필요
수정판 · "hello, world!" (13바이트)	COM6에 같은 문자열	COM6: hello, world! · MCU: 13바이트, 전송 실패 0	일치
수정판 · "holy moly" 등 여러 묶음	묶음마다 같은 문자열	COM6에 holy molyhello, world! 순서대로 이어서 표시	일치



첫 판 · 왼쪽 COM5(MCU)는 1, A를 하나씩 보낸 기록, 오른쪽 COM6(FPGA)는 받은 1A



수정판 · 왼쪽 COM5에 "hello, world!" (13바이트), 오른쪽 COM6에 FPGA가 받은 문자열이 그대로 표시

과제6 첫 판 영상 (10.2 s) · 한 글자씩

영상: LAB6/evidence/videos/LAB6-04_task6_single.mp4 (GitHub에서 열기 ↗)

과제6 수정판 영상 (23.2 s) · 여러 글자 묶음

영상: LAB6/evidence/videos/LAB6-04_task6_uart_fpga.mp4 (GitHub에서 열기 ↗)

⑥ 문제 해결 과정 — FPGA가 아무것도 받지 못함

처음에는 MCU의 전송 결과가 0인데도 FPGA COM6에 0바이트가 오거나, 의미 없는 바이트가 무작위로 왔다. 오실로스코프로 보면 PC10 파형은 정상이었다. 원인을 하나씩 좁힌 과정은 다음과 같다.

단계	확인한 것	결과 · 판단
1	COM6이 PC에 보이지 않음	FTDI 드라이버의 "Load VCP"가 꺼져 있었다. 장치 관리자에서 켜자 COM6 생성. JTAG(Xilinx Platform Cable)은 별도 장치라 비트스트림은 정상으로 올라갔다.
2	PC10 파형 (오실로스코프)	`'U'` (0x55)를 반복 전송해 주파수 4.771 kHz 측정 → 비트 폭 $1/(2 \times 4771) = 104.8\ \mu s$, 약 9542 bps. 9600 bps 대비 −0.6 %로 정상 범위.
3	CLOCK SELECT 다이얼	50 MHz 위치가 아니었다. FPGA 수신기의 비트 폭 계산(5208클록)은 50 MHz를 전제로 하므로, 다이얼을 맞춘 뒤 UART가 맞기 시작했다.
4	PORT A ENABLE 스위치 (DIP SW5의 2번)	ON이면 LED에 엉뚱한 값이 무작위로 바뀌고, OFF로 바꾸자 정확한 값이 왔다. 두 번 바꿔 보아 같은 결과를 확인했다. 확장 포트와 FPGA 사이의 버스 스위치(PI3C3245)는 활성 LOW 입력(풀다운 4.7 kΩ)이라 **OFF = 연결**이다. ON이면 F8이 떠(floating) 잡음을 바이트로 읽었다.

4. STM32_2 과제7 — UART 명령으로 FPGA LED 제어

① 목적

FPGA가 받은 글자를 명령으로 해석해 LED1을 제어한다: `'0'` 끄기, `'1'` 켜기, `'2'` 반전, 그 밖의 글자는 그대로 둔다. "켜기"를 반복하면 계속 켜져 있고 "반전"을 반복하면 매번 바뀌는지 확인한다. LED2 ~ 8에는 마지막으로 받은 바이트의 아래 7비트를 표시한다.

② 이론

$$lamp_{next} = \begin{cases} 0 & rx = \text{0x30 ('0')} \\ 1 & rx = \text{0x31 ('1')} \\ \overline{lamp} & rx = \text{0x32 ('2')} \\ lamp & \text{그 밖} \end{cases}$$

켜기는 현재 상태와 상관없이 1로 정하고, 반전은 현재 상태를 뒤집는다. 판단은 FPGA가 한다. MCU는 과제6 수정판과 같은 방식으로 모든 바이트를 넘기고, 한 묶음이 끝나면 "이 명령들을 차례로 적용하면 LED1이 어떻게 되어야 하는지"를 참고로 출력한다.

③ 코드 (과제6 수정판과 다른 부분)

```
static uint8_t expect_led = 0U;   /* FPGA 리셋 직후 0이라고 가정 */
...
    if (data == '0') expect_led = 0U;
    else if (data == '1') expect_led = 1U;
    else if (data == '2') expect_led ^= 1U;
...
    printf("명령 \"%s\" (%lu바이트) -> 예상 LED1: %s  (전송 실패 %lu, 버퍼 넘침 %lu)\r\n",
           line, (unsigned long)count, expect_led ? "켜짐" : "꺼짐",
           (unsigned long)fail, (unsigned long)rx_overflow);
```

④ 사전 검증

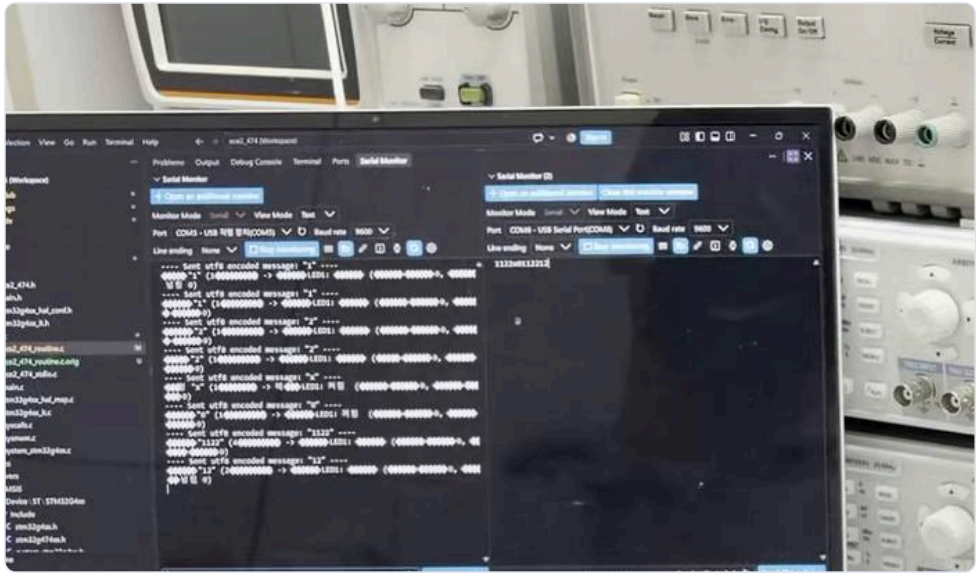
PC 모의 실행 출력과, 그 바이트를 연속으로 보낸 FPGA TB 결과(LED1 = 참조 모델):

```
명령 "111" (3바이트) -> 예상 LED1: 켜짐  (전송 실패 0, 버퍼 넘침 0)
명령 "222" (3바이트) -> 예상 LED1: 꺼짐  (전송 실패 0, 버퍼 넘침 0)
명령 "0x0" (3바이트) -> 예상 LED1: 꺼짐  (전송 실패 0, 버퍼 넘침 0)
명령 "1122" (4바이트) -> 예상 LED1: 켜짐  (전송 실패 0, 버퍼 넘침 0)
```

```
  2006.2 us  PC(FPGA COM) <- 0x31   LED1=1  LED2~8=0110001
  3054.2 us  PC(FPGA COM) <- 0x31   LED1=1  LED2~8=0110001
  4102.2 us  PC(FPGA COM) <- 0x31   LED1=1  LED2~8=0110001
  5150.2 us  PC(FPGA COM) <- 0x32   LED1=0  LED2~8=0110010
  6198.2 us  PC(FPGA COM) <- 0x32   LED1=1  LED2~8=0110010
  7246.2 us  PC(FPGA COM) <- 0x32   LED1=0  LED2~8=0110010
  8294.2 us  PC(FPGA COM) <- 0x30   LED1=0  LED2~8=0110000
  9342.2 us  PC(FPGA COM) <- 0x78   LED1=0  LED2~8=1111000
 10390.2 us  PC(FPGA COM) <- 0x30   LED1=0  LED2~8=0110000
 11438.2 us  PC(FPGA COM) <- 0x31   LED1=1  LED2~8=0110001
 12486.2 us  PC(FPGA COM) <- 0x31   LED1=1  LED2~8=0110001
 13534.2 us  PC(FPGA COM) <- 0x32   LED1=0  LED2~8=0110010
 14582.2 us  PC(FPGA COM) <- 0x32   LED1=1  LED2~8=0110010
PASS uart_cmd_top: 13 bytes echoed, LED1 follows 0/1/2 commands
sim/tb_b2b.sv:46: $finish called at 14902600000 (1ps)
```

⑤ 보드 검증

입력 (묶음)	기대 LED1	관찰	결과
'1'을 반복	계속 켜짐	켜진 채 유지	일치
'2'를 반복	보낼 때마다 꺼짐 ↔ 켜짐	매번 반전	일치
'x' (명령 아님)	그대로	변화 없음 (LED2~8만 바뀜)	일치
'0'	꺼짐	꺼짐	일치
"1122" · "12" (여러 글자 묶음)	차례로 적용: 켜짐 → 켜짐 → 꺼짐 → 켜짐 / 켜짐 → 꺼짐	COM6에 1122, 12가 그대로 도착하고 LED1은 마지막 결과와 같음	일치



과제7 · 왼쪽 COM5에 보낸 명령(1, 2, x, 0, 1122, 12)과 예상 LED1, 오른쪽 COM6에 FPGA가 받은 명령 바이트가 보낸 순서대로 이어서 표시

과제7 영상 (24.3 s) · 명령마다 보드의 LED1 변화

영상: LAB6/evidence/videos/LAB6-05_task7_led_cmd.mp4 (GitHub에서 열기 ↗)

5. FPGA 실습 ① 서보모터 제어 (servo_ctrl)

① 목적

RC 서보모터는 20 ms마다 오는 펄스의 HIGH 길이로 각도를 정한다(1.0 ms ~ 2.0 ms). 키패드 4 · 6을 누를 때마다 폭을 0.1 ms씩 바꾸어 서보가 단계적으로 도는지 확인한다.

② 이론

$$N_{frame} = \frac{50\ \text{MHz}}{50\ \text{Hz}} = 1\,000\,000\ \text{클록}, \qquad N_{width} = t_{\mu s} \times 50 \quad (1.5\ \text{ms} \rightarrow 75\,000\ \text{클록})$$

수업 식 `pulse_us*(CLK_HZ/1000)/1000`은 50 MHz에서 타이밍을 어겼으므로(WNS −0.609 ns), 보드용 `servo_pwm_board`에서 ×50 상수 곱으로 바꾸었다(실험 전 보고서 1장). 버튼은 10 ms 디바운스 후 한 번에 한 클록 펄스로 만든다.

③ 포트 · 보드 매핑과 교재 대조 (교재 실습 25 핀표)

포트	교재 핀 · 장치	XDC	대조
clk_50mhz	B6 · 메인 클록	B6	일치
rst_p	Y1 · DIP SW1	Y1	일치
btn_left / btn_right	N1 · N6 (키패드 4 · 6)	N1 · N6	일치
servo	**AA22** · SERVO	AA22	임시 E8에서 변경
led[7] / led[3:0]	L4 / M3 M1 N5 … (LED1 · LED5~8)	같음	일치

④ XDC

```
# Combo II-DLD S75 / LAB6-01 servo_board_top
# 확인됨: clk B6, DIP SW1 Y1, keypad 4 N1 / 6 N6, LED1~8.  servo = AA22 (교재 실습 25 핀표, 2026-10-26 실험에서 E8 -> AA22)
set_property PACKAGE_PIN B6 [get_ports clk_50mhz]
set_property PACKAGE_PIN Y1 [get_ports rst_p]
set_property PACKAGE_PIN N1 [get_ports btn_left]
```

```
set_property PACKAGE_PIN N6 [get_ports btn_right]
set_property PACKAGE_PIN AA22 [get_ports servo]
set_property PACKAGE_PIN N5 [get_ports {led[0]}]
set_property PACKAGE_PIN M1 [get_ports {led[1]}]
set_property PACKAGE_PIN M3 [get_ports {led[2]}]
set_property PACKAGE_PIN M7 [get_ports {led[3]}]
set_property PACKAGE_PIN N7 [get_ports {led[4]}]
set_property PACKAGE_PIN M2 [get_ports {led[5]}]
set_property PACKAGE_PIN M4 [get_ports {led[6]}]
set_property PACKAGE_PIN L4 [get_ports {led[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports *]
create_clock -name clk_50mhz -period 20.000 [get_ports clk_50mhz]
set_false_path -from [get_ports {rst_p btn_left btn_right}]
set_false_path -to [get_ports {servo led[*]}]
```

⑤ 보드 빌드 결과

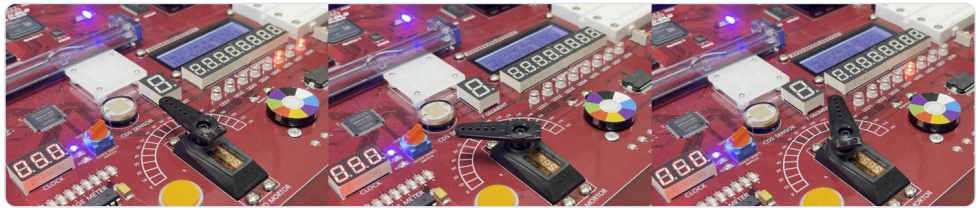
최상위	LUT	FF	IOB	WNS	WHS	비트스트림
servo_board_top + servo_pwm_board (AA22)	411	125	13	+13.281 ns	+0.099 ns	생성됨

실험 전(E8)의 WNS +12.860 ns와 조금 다른 것은 핀 위치가 바뀌어 배치 · 배선이 달라졌기 때문이다. 자원 수는 같다.

⑥ 보드 검증

확인 항목	기대	관찰	결과
DIP SW1 ON → OFF (리셋 해제)	가운데 (1.5 ms)	서보 혼이 가운데 위치	일치
키패드 4를 여러 번	한 번에 한 칸(0.1 ms)씩 한쪽으로, 1.0 ms에서 멈춤	누를 때마다 한 단계씩 회전, 끝에서 멈춤	일치
키패드 6을 여러 번	반대쪽으로 한 칸씩, 2.0 ms에서 멈춤	반대 방향으로 단계 회전, 끝에서 멈춤	일치

오실로스코프 측정은 생략했다. 실험 전 보고서에서 20 ms 프레임과 1.0/1.5/2.0 ms 폭을 두 시뮬레이터로 확인했고, 보드에서는 버튼 한 번에 한 단계씩 움직이는 것(디바운스 동작)과 양 끝에서 멈추는 것을 눈으로 확인했다.



서보 · 키패드를 누를 때마다 서보 혼의 각도가 단계적으로 바뀜 (8.2 s 영상의 세 장면)

영상: LAB6/evidence/videos/LAB6-06_servo.mp4 (GitHub에서 열기 ↗)

6. FPGA 실습 ② SRAM 읽기 · 쓰기 (sram)

① 목적

외부 SRAM 칩의 주소 0 ~ 15에 DIP 스위치로 정한 4비트 값을 쓰고, 같은 주소에서 다시 읽어 LED5 ~ 8로 확인한다. FPGA와 SRAM이 16비트 데이터선을 번갈아 쓰는 방법, 즉 쓸 때는 FPGA가 값을 내고 읽을 때는 손을 떼는(Z) 방법을 익힌다.

② 이론

CS · WE · OE는 모두 LOW 활성이다. 쓰기: CS = 0, WE = 0, FPGA가 데이터선 구동. 읽기: CS = 0, OE = 0, FPGA는 Z, SRAM이 구동. 두 버튼을 함께 누르면 둘 다 무시한다.

보드의 칩은 Samsung **K6R4016V1D**(256K × 16, 3.3 V)이다. 데이터시트에 따르면 쓰기는 "CS나 WE 중 먼저 HIGH가 되는 순간" 끝나고 그때 데이터선 값이 저장되며, 데이터 홀드 시간 t_{DH}의 최소값은 0 ns이다[2]. 실험 전에 ISSI 칩 모델로 검토한 결론(교재 회로는 WE 상승과 데이터 Z가 같은 순간이라 여유 0)이 이 칩에도 그대로 적용된다. 보드용 최상위는 WE를 올린 뒤 데이터를 2클록(40 ns) 더 유지한다.

$$t_{DH}^{\text{교재}} = 0\ \text{ns}\ (\text{경쟁}), \qquad t_{DH}^{\text{보드용}} = 2 \times 20\ \text{ns} = 40\ \text{ns} \ \geq\ 0\ \text{ns}$$

③ 포트 · 보드 매핑과 교재 대조 (교재 p.603 핀 구성표)

포트	교재 핀	XDC	대조
ram_address[0:17]	L20 L22 M17 M18 N18 W22 V22 V20 V18 U20 V19 U22 U19 T21 T19 L19 L16 K19	같음 (18줄)	작성
ram_data[0:15]	P17 P21 R16 R18 R20 T17 T20 T22 T16 R19 R17 P22 P20 P16 N19 M16	같음 (16줄)	작성
ram_cs / ram_we / ram_oe	N20 / U17 / L21	같음	작성
ram_ub / ram_lb	표에 없음 (보드에서 GND 고정)	포트 삭제	최상위 수정
ctrl_w / ctrl_r	K4 / N8 (키패드 1 · 2)	같음	일치
address · data_in	DIP SW1~4 · SW5~8	같음	일치
clk_50mhz (보드용)	B6	B6	일치

IOB는 56개 = 입력 11(클록 · 버튼 2 · DIP 8) + LED 8 + SRAM 37이다. 실험 전 합성의 58개에서 UB/LB 두 개가 빠졌다.

④ 보드용 최상위 · XDC

```
`timescale 1ns/1ps
// 보드 시연용 SRAM 최상위.
// 수업 sram은 조합회로라서 쓰기 버튼을 떼는 순간 WE 상승과 데이터 버스 해제(Z)가 동시에 일어난다
// -> SRAM 데이터 홀드 tHD = 0 ns 조건과 여유 0의 경쟁. 여기서는 50 MHz 클록으로
//   (1) 버튼을 2단 동기화하고, (2) 쓰기가 끝난 뒤 데이터를 HOLD_CYCLES 클록 더 구동한 다음 Z로 놓는다.
// 동시 요청 무시, 미사용 시 Z, LED 표시는 수업 sram과 같다.
// 보드(K6R4016V1D)의 UB/LB(39·40번 핀)는 회로도상 GND에 직결되어 있어 FPGA 포트가 없다(교재 p.603).
module sram_board_top #(parameter integer HOLD_CYCLES = 2)(
    input  wire clk_50mhz,
    input  wire ctrl_w, ctrl_r,
    input  wire [3:0] address, data_in,
    output wire ram_cs, ram_oe, ram_we,
    output wire [17:0] ram_address,
    inout  wire [15:0] ram_data,
```

```
    output wire [7:0] led_out
);
    (* ASYNC_REG = "TRUE" *) reg [1:0] w_sync = 2'b00, r_sync = 2'b00;
    reg [3:0] hold = 0;
    always @(posedge clk_50mhz) begin
        w_sync <= {w_sync[0], ctrl_w};
        r_sync <= {r_sync[0], ctrl_r};
    end
    wire drive_req, read_req, cs_unused, oe_unused, we_unused;
    // 수업 버스 제어 모듈을 동기화된 버튼으로 그대로 사용(동시 요청 무시 규칙 유지)
    sram_bus_control u_control(w_sync[1], r_sync[1], drive_req, read_req, cs_unused, oe_unused,
we_unused);
    always @(posedge clk_50mhz)
        if (drive_req) hold <= HOLD_CYCLES;
        else if (hold != 0) hold <= hold - 1'b1;
    wire drive_data = drive_req || (hold != 0);          // 쓰기 끝난 뒤에도 데이터 유지
    wire read_data  = read_req && (hold == 0);           // 데이터 해제 후에만 읽기
    assign ram_we = !drive_req;
    assign ram_oe = !read_data;
    assign ram_cs = !(drive_data || read_data);
    assign ram_address = {14'b0, address};
    assign ram_data = drive_data ? {12'b0, data_in} : 16'bz;
    assign led_out = read_data ? {4'b0, ram_data[3:0]} : 8'b0;
endmodule
```

```
# Combo II-DLD S75 / LAB6-02 sram_board_top
# clk_50mhz = B6 (sram_board_top)
# 확인됨: keypad 1 K4 = ctrl_w(쓰기), keypad 2 N8 = ctrl_r(읽기), DIP SW1~4 = address[3:0], SW5~8 =
data_in[3:0], LED1~8.
set_property PACKAGE_PIN K4 [get_ports ctrl_w]
set_property PACKAGE_PIN N8 [get_ports ctrl_r]
set_property PACKAGE_PIN Y1 [get_ports {address[3]}]
set_property PACKAGE_PIN W3 [get_ports {address[2]}]
set_property PACKAGE_PIN U2 [get_ports {address[1]}]
set_property PACKAGE_PIN T1 [get_ports {address[0]}]
set_property PACKAGE_PIN W4 [get_ports {data_in[3]}]
set_property PACKAGE_PIN W1 [get_ports {data_in[2]}]
set_property PACKAGE_PIN V4 [get_ports {data_in[1]}]
set_property PACKAGE_PIN U4 [get_ports {data_in[0]}]
set_property PACKAGE_PIN L4 [get_ports {led_out[7]}]
set_property PACKAGE_PIN M4 [get_ports {led_out[6]}]
set_property PACKAGE_PIN M2 [get_ports {led_out[5]}]
set_property PACKAGE_PIN N7 [get_ports {led_out[4]}]
set_property PACKAGE_PIN M7 [get_ports {led_out[3]}]
set_property PACKAGE_PIN M3 [get_ports {led_out[2]}]
set_property PACKAGE_PIN M1 [get_ports {led_out[1]}]
set_property PACKAGE_PIN N5 [get_ports {led_out[0]}]
set_property PACKAGE_PIN B6 [get_ports clk_50mhz]
# SRAM K6R4016V1D (교재 p.603 핀 구성표). UB/LB는 보드에서 GND 직결
set_property PACKAGE_PIN N20 [get_ports ram_cs]
set_property PACKAGE_PIN U17 [get_ports ram_we]
set_property PACKAGE_PIN L21 [get_ports ram_oe]
set_property PACKAGE_PIN L20 [get_ports {ram_address[0]}]
set_property PACKAGE_PIN L22 [get_ports {ram_address[1]}]
set_property PACKAGE_PIN M17 [get_ports {ram_address[2]}]
set_property PACKAGE_PIN M18 [get_ports {ram_address[3]}]
set_property PACKAGE_PIN N18 [get_ports {ram_address[4]}]
set_property PACKAGE_PIN W22 [get_ports {ram_address[5]}]
set_property PACKAGE_PIN V22 [get_ports {ram_address[6]}]
set_property PACKAGE_PIN V20 [get_ports {ram_address[7]}]
set_property PACKAGE_PIN V18 [get_ports {ram_address[8]}]
set_property PACKAGE_PIN U20 [get_ports {ram_address[9]}]
set_property PACKAGE_PIN V19 [get_ports {ram_address[10]}]
set_property PACKAGE_PIN U22 [get_ports {ram_address[11]}]
set_property PACKAGE_PIN U19 [get_ports {ram_address[12]}]
set_property PACKAGE_PIN T21 [get_ports {ram_address[13]}]
set_property PACKAGE_PIN T19 [get_ports {ram_address[14]}]
set_property PACKAGE_PIN L19 [get_ports {ram_address[15]}]
set_property PACKAGE_PIN L16 [get_ports {ram_address[16]}]
set_property PACKAGE_PIN K19 [get_ports {ram_address[17]}]
set_property PACKAGE_PIN P17 [get_ports {ram_data[0]}]
set_property PACKAGE_PIN P21 [get_ports {ram_data[1]}]
set_property PACKAGE_PIN R16 [get_ports {ram_data[2]}]
set_property PACKAGE_PIN R18 [get_ports {ram_data[3]}]
set_property PACKAGE_PIN R20 [get_ports {ram_data[4]}]
set_property PACKAGE_PIN T17 [get_ports {ram_data[5]}]
set_property PACKAGE_PIN T20 [get_ports {ram_data[6]}]
set_property PACKAGE_PIN T22 [get_ports {ram_data[7]}]
set_property PACKAGE_PIN T16 [get_ports {ram_data[8]}]
set_property PACKAGE_PIN R19 [get_ports {ram_data[9]}]
set_property PACKAGE_PIN R17 [get_ports {ram_data[10]}]
set_property PACKAGE_PIN P22 [get_ports {ram_data[11]}]
set_property PACKAGE_PIN P20 [get_ports {ram_data[12]}]
set_property PACKAGE_PIN P16 [get_ports {ram_data[13]}]
set_property PACKAGE_PIN N19 [get_ports {ram_data[14]}]
set_property PACKAGE_PIN M16 [get_ports {ram_data[15]}]
set_property IOSTANDARD LVCMOS33 [get_ports *]
create_clock -name clk_50mhz -period 20.000 [get_ports clk_50mhz]
set_false_path -from [get_ports {ctrl_w ctrl_r address[*] data_in[*]}]
set_false_path -to [get_ports {ram_* led_out[*]}]
set_false_path -from [get_ports {ram_data[*]}]
```

⑤ 시뮬레이션 (UB/LB를 뺀 최상위로 다시 실행)

보드 TB에서 SRAM 모델의 UB/LB 입력을 `1'b0` 으로 고정하고 Icarus로 다시 실행했다. 교재 회로는 17번 쓰기 모두 경쟁(읽기 16/16 틀림), 보드용은 경쟁 · 위반 · 버스 충돌 0으로 실험 전과 같다. (XSim 결과 파일은 실험 전, UB/LB 포트가 있던 최상위로 실행한 것이다.)

```
CLASS sram     : writes=17, tHD=0 races=17, wrong reads=16/16
BOARD sram_top : writes=17, tHD=0 races=0, violations=0, contention=0
PASS SRAM board: 17 writes, 17 reads, overlap idle, data held after WE rise, no bus contention
sim/tb_board.sv:36: $finish called at 120100000 (1ps)
```

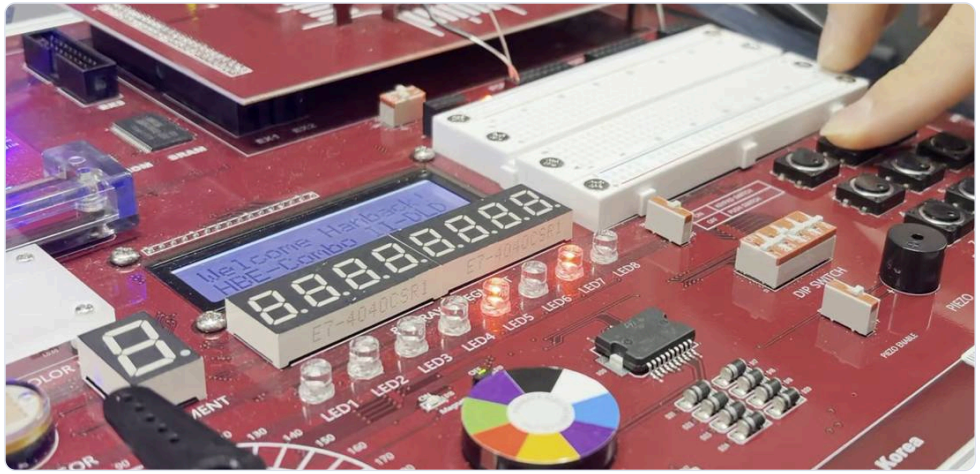
⑥ 보드 빌드 결과

최상위	LUT	FF	IOB	WNS	WHS	비트스트림
sram_board_top	11	8	56	+16.899 ns	+0.160 ns	생성됨

처음 빌드는 비트스트림이 만들어지지 않았다(bit = 0, IOB 58). PC로 보낸 XDC · 최상위 파일이 이전 내용 그대로였기 때문으로, 파일을 다시 보내고 크기를 확인한 뒤 다시 빌드해 IOB 56 · 비트스트림 생성을 확인했다. 입력 버튼 · DIP와 SRAM · LED 출력은 사람 속도로 바뀌는 경로라 `set_false_path` 로 시간 검사에서 뺐다.

⑦ 보드 검증

확인 항목	기대	관찰	결과
주소 · 값을 DIP로 정하고 키패드 1(쓰기)	LED 변화 없음 (쓰기만)	LED 꺼진 채	일치
같은 주소에서 키패드 2(읽기)를 누르는 동안	LED5~8 = 쓴 값	쓴 값이 LED5~8에 표시	일치
다른 주소에 다른 값 쓰고 번갈아 읽기	주소마다 자기 값	주소를 바꾸면 각자의 값이 나옴	일치
키패드 1 · 2를 함께 누름	LED 꺼짐 (무시)	LED 꺼짐	일치



SRAM · 읽기 버튼을 누르는 동안 LED5~8 중 두 개가 켜져 저장된 4비트 값을 표시 (41.6 s 영상)

영상: LAB6/evidence/videos/LAB6-07_sram.mp4 (GitHub에서 열기 ↗)

7. FPGA 실습 ③ DAC (dac)

① 목적

8비트 병렬 DAC 칩에 0, 1, ..., 255, 0, ...을 10 ms마다 차례로 써서 계단 모양의 톱니파를 만들고, 오실로스코프로 디지털 값이 아날로그 전압으로 바뀌는 것을 확인한다.

② 이론

$$V_{out} = V_{ref} \cdot \frac{D}{256}, \qquad T_{ramp} = 256 \times \frac{500\,000}{50\ \mathrm{MHz}} = 256 \times 10\ \mathrm{ms} = 2.56\ \mathrm{s}$$

보드용 `dac_board_top`은 쓰기 펄스(WR LOW)를 2클록(40 ns)으로 늘려 AD7302의 최소 20 ns 조건에 여유를 두었다(실험 전 보고서 3장).

③ 포트 · 보드 매핑과 교재 대조 (교재 p.625)

포트	교재 핀 · 장치	XDC	대조
clk_50mhz	N8로 표기 (키패드 2 핀)	B6	**B6로 정정**
rst_p · add_sel	K4 (키패드 1) · Y1 (DIP SW1)	K4 · Y1	일치
dac_d[7:0]	B3 B1 C4 C2 D5 D2 E5 E3	같음	일치
dac_a_b · dac_csn · dac_ldacn · dac_wen	G2 · E1 · F4 · G4	같음	일치
led_out[7:0]	LED1~8	같음	일치

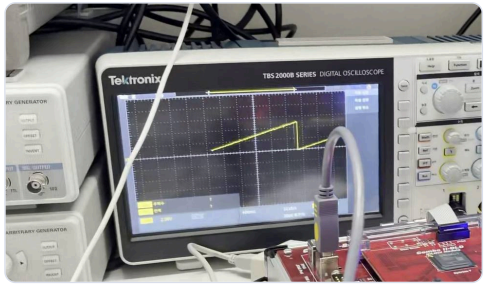
④ 보드 빌드 결과

최상위	LUT	FF	IOB	WNS	WHS	비트스트림
dac_board_top (실험 전 빌드 그대로)	22	62	23	+14.854 ns	+0.165 ns	실험 전 생성

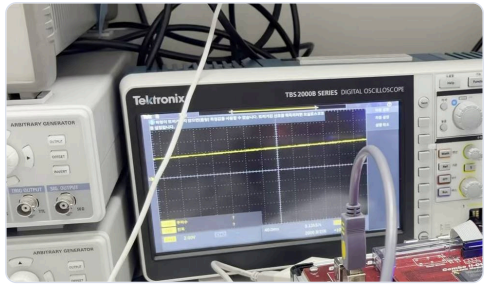
⑤ 보드 검증

확인 항목	기대	관찰	결과
D/A OUT 2 · 2 V/칸 · 400 ms/칸	서서히 오르다 한 번에 0으로 떨어지는 톱니, 주기 2.56 s	오르는 직선 뒤 수직으로 떨어지는 톱니. 상승 구간이 약 6칸 남짓(≈ 2.5 s)	일치
시간축 40 ms/칸으로 확대	10 ms마다 Vref/256씩 오르는 계단 (칸당 4계단)	거의 수평에 가까운 천천히 오르는 선	일치
LED1~8	현재 값 D를 2진수로 표시, 10 ms마다 증가	LED가 2진수로 세어 올라감	일치

처음에는 시간축이 2 ms/칸이라 화면 전체(20 ms)가 계단 두 개 길이밖에 안 되어 선이 "계속 올라가기만" 하는 것처럼 보였다. 한 주기가 2.56 s이므로 화면 10칸에 한 주기 이상이 들어가도록 시간축을 수백 ms/칸으로 늘리자 톱니 전체가 보였다. 진폭은 커서로 정밀하게 재지 않았다. 계단 한 칸(Vref/256 ≈ 십 mV 정도)은 2 V/칸 화면에서는 구분되지 않는다.



D/A OUT 2 · 2 V/칸, 400 ms/칸 · 오르다 떨어지는 톱니



40 ms/칸으로 확대 · 한 주기의 일부만 보여 거의 평평하게 천천히 오름

DAC 영상 (29.6 s) · 오실로스코프 톱니파와 시간축 변경

영상: LAB6/evidence/videos/LAB6-08_dac.mp4 (GitHub에서 열기 ↗)

8. 고찰

1. **"전송 결과 0"은 통신 성공이 아니다.** 예제 ② · 과제6 내내 MCU는 결과 0(HAL_OK)을 냈지만, FPGA는 0바이트를 받거나 잡음을 받았다. UART에는 수신 확인이 없으므로, 받는 쪽의 결과(FPGA가 되돌려 보낸 COM6 출력과 LED)로만 성공을 판단할 수 있었다. 과제6을 "FPGA가 받은 글자를 다시 PC로 보내는" 구조로 만든 것이 원인을 찾는 데 결정적이었다.

2. **보드 설정도 회로의 일부다.** 같은 비트스트림 · 같은 코드인데 결과를 바꾼 것은 CLOCK SELECT 다이얼(50 MHz)과 PORT A ENABLE 스위치(OFF = 연결)였다. 실험 전 보고서는 ENABLE을 ON으로 적었는데, 이름만 보고 판단한 것이었다. 버스 스위치의 활성 LOW 입력처럼 스위치의 의미는 회로도로 확인해야 한다. 원인을 좁힐 때는 오실로스코프로 MCU 쪽 파형이 정상(9542 bps, 3.2 V)임을 먼저 확인해, 문제를 FPGA 쪽 연결로 한정할 수 있었다.
3. **기다리는 입력(블로킹)과 인터럽트 입력.** getchar 방식은 한 글자를 처리하는 동안(printf ≈ 42 ms) 다음 글자를 받을 수 없어, 1 ms 간격으로 이어 오는 문자열을 놓쳤다. 실험 전 보고서 고찰 6에서 예상한 한계가 조교의 여러 글자 요구로 그대로 드러났다. 수신 인터럽트가 바이트를 링 버퍼에 넣기만 하고 바로 돌아오게 하자, 처리 시간이 길어도 입력을 잃지 않았다. 출력은 20 ms 동안 입력이 없을 때 한 번만 해서, 출력이 다음 입력과 겹치는 시간도 줄였다.
4. **클록이 서로 다른 두 장치의 UART.** 실측 MCU 비트 폭 104.8 µs(9542 bps)와 FPGA 수신기 9600.6 bps는 0.6 % 차이다. 글자 하나(10비트) 동안 쌓이는 어긋남은 6 %의 비트 폭으로 허용치(반 비트, 50 %)보다 훨씬 작고, 시작 비트마다 다시 맞추므로 연속 전송에서도 쌓이지 않는다. 연속 14바이트 TB가 이것을 확인했다.
5. **교재 핀표는 신호 이름과 칩 기준으로 읽는다.** 서보는 임시 핀(E8)을 교재 핀(AA22)으로, SRAM은 p.603 표로 37핀을 채웠다. 표에 UB/LB가 없다는 것은 보드에서 GND에 고정되었다는 뜻이므로, 최상위에서 포트를 지웠다. DAC 표의 클록 N8은 LAB5에서처럼 키패드 핀이라 B6로 바로잡았다. 또 PC로 보낸 파일이 실제로 바뀌었는지(크기) 확인하지 않아 빌드를 한 번 날렸다. 빌드 전에 입력 파일을 확인하는 습관이 필요하다.
6. **오실로스코프 시간축은 신호의 주기에 맞춘다.** DAC 톱니는 주기 2.56 s라 2 ms/칸에서는 한 계단도 다 못 보아 "계속 올라가기만" 보였다. 반대로 UART 비트(104 µs)는 4 µs/칸에서 에지 하나만 보였다. 먼저 계산한 주기로 시간축을 정하면(주기 ÷ 10칸) 한 번에 원하는 모양을 볼 수 있다.
7. **화면 표시와 실제 데이터를 구분한다.** Serial Monitor의 한글 깨짐은 9600 bps에서 UTF-8 3바이트가 나뉘어 들어와 생긴 표시 문제였다. 같은 출력이 PC 모의 실행에서는 정상이고, 영문 · 16진수 부분은 맞게 보였으므로 MCU 코드 문제가 아님을 알 수 있었다.

참고문헌

1. [1] D. M. Harris and S. L. Harris, *Digital Design and Computer Architecture*, 2nd ed. Waltham, MA, USA: Morgan Kaufmann, 2013. ISBN 978-0-12-394424-5. — Ch. 3.5 타이밍(셋업 · 홀드), Ch. 4 삼상태 버퍼, Ch. 8.6 I/O의 UART · 인터럽트 · PWM · DAC.
2. [2] Samsung Electronics, *K6R4016V1D: 256Kx16 Bit High Speed Static RAM (3.3V Operating)*, Data Sheet, Rev. 4.0, Mar. 2004, xdevs.com (확인일 2026-10-26). — 쓰기 시간 조건(-10: tWP 7 ns, tDW 5 ns, tDH 0 ns), "쓰기는 CS 또는 WE 중 먼저 HIGH가 되는 순간 끝남", UB/LB 바이트 선택.
3. [3] STMicroelectronics, *stm32g4xx-hal-driver*, GitHub, https://github.com/STMicroelectronics/stm32g4xx-hal-driver, 태그 v1.2.7. — `stm32g4xx_hal_uart.c`의 `HAL_UART_Receive_IT` · `HAL_UART_RxCpltCallback` · `HAL_UART_ErrorCallback` (오류 시 수신 중단).
4. [4] Analog Devices, *AD7302 Data Sheet*. — WR 펄스 폭 20 ns, 데이터 셋업 · 홀드, LDAC LOW일 때 WR 상승에서 출력 갱신.

수업 자료(인용 출처 수에 넣지 않음): Glaysia/ece2_474 v1.1.0의 docs/STM32_2.md · docs/FPGA_additional.md, 실습 교재 「Combo II DLD 장비를 이용한 디지털 논리 회로 설계」 p.73(확장 포트 BDIO) · p.603(SRAM 핀 구성표) · p.625(DAC 핀표) · 실습 25(서보 핀), LAB6 실험 전 보고서(태그 lab6-prelab).